Computer and Network Security (2026-2027)

Part 4: Asymmetric Encryption

Jeroen Famaey
jeroen.famaey@uantwerpen.be

University of Antwerp
Faculty of Science

We want your feedback on the labs!



https://forms.cloud.microsoft/e/SUAcsbYQzz

University of Antwerp
Faculty of Science

Part 4 – Topics

1. Asymmetric Encryption Basics
2. Rivest-Shamir-Adleman (RSA)
3. ElGamal cryptography
4. Digital Signatures

Asymmetric Encryption Basics

Some facts about asymmetric encryption

- Also known as **public key cryptography**, as it uses a public and private (i.e., secret) key
- Based on mathematical functions, not substitution and permutation
- Addresses two concerns with symmetric encryption
  - Secret keys need to be distributed using a trusted key distribution center
  - It does not support digital signatures
- Idea first made public by Diffie and Hellman in 1976

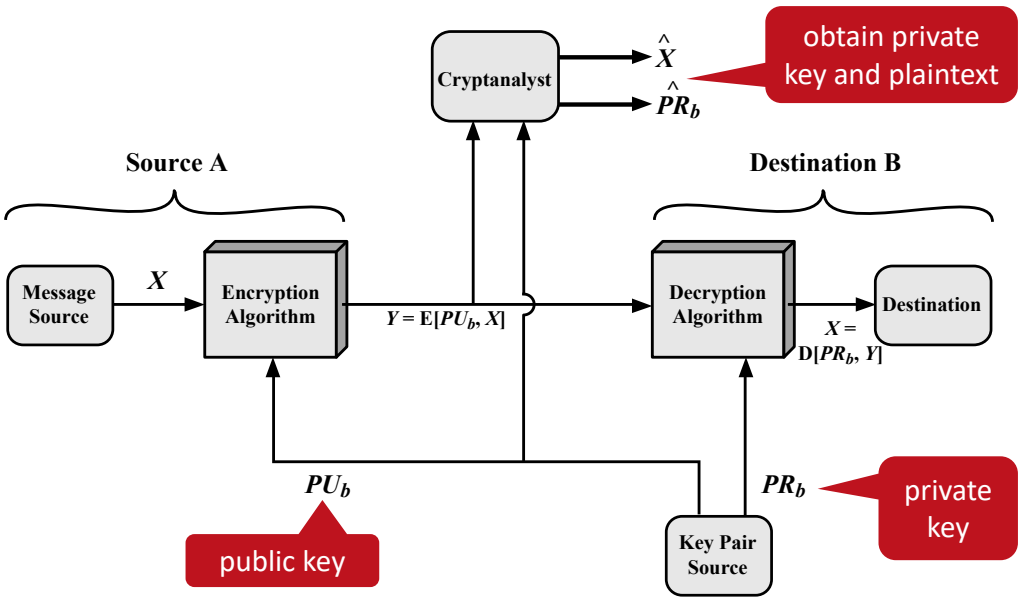


Public-key cryptography provides a radical departure from all that has gone before. For one thing, public-key algorithms are based on mathematical functions rather than on substitution and permutation. More important, public-key cryptography is asymmetric, involving the use of two separate keys, in contrast to symmetric encryption, which uses only one key. The use of two keys has profound consequences in the areas of confidentiality, key distribution, and authentication, as we shall see.

Misconceptions about asymmetric encryption

Statement	True or false?	Comments
Public-key encryption is more secure than private key encryption		
Public-key encryption has made symmetric encryption obsolete		
Key distribution is trivial in public-key encryption		

Confidentiality using asymmetric encryption



University of Antwerp | Faculty of Science

8

Let us take a closer look at the essential elements of a public-key encryption scheme. There is some source A that produces a message in plaintext, $X = [X_1, X_2, \ldots, X_M]$. The M elements of X are letters in some finite alphabet. The message is intended for destination B. B generates a related pair of keys: a public key, PU_b, and a private key, PR_b. PR_b is known only to B, whereas PU_b is publicly available and therefore accessible by A. With the message X and the encryption key PU_b as input, A forms the ciphertext $Y = [Y_1, Y_2, \ldots, Y_N]$:

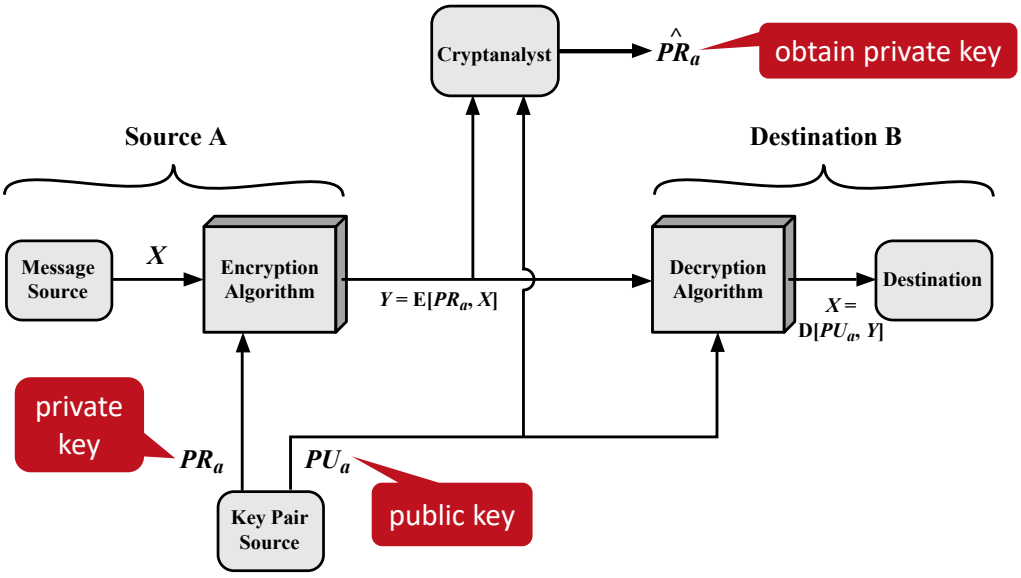
$$Y = E(PU_b, X)$$

The intended receiver, in possession of the matching private key, is able to invert the transformation:

$$X = D(PR_b, Y)$$

An adversary, observing Y and having access to PU_b, but not having access to PR_b or X, must attempt to recover X and/or PR_b. It is assumed that the adversary does have knowledge of the encryption (E) and decryption (D) algorithms. If the adversary is interested only in this particular message, then the focus of effort is to recover X by generating a plaintext estimate $\hat{X}$. Often, however, the adversary is interested in being able to read future messages as well, in which case an attempt is made to recover PR_b by generating an estimate $\widehat{PR}_b$.

Authentication using asymmetric encryption



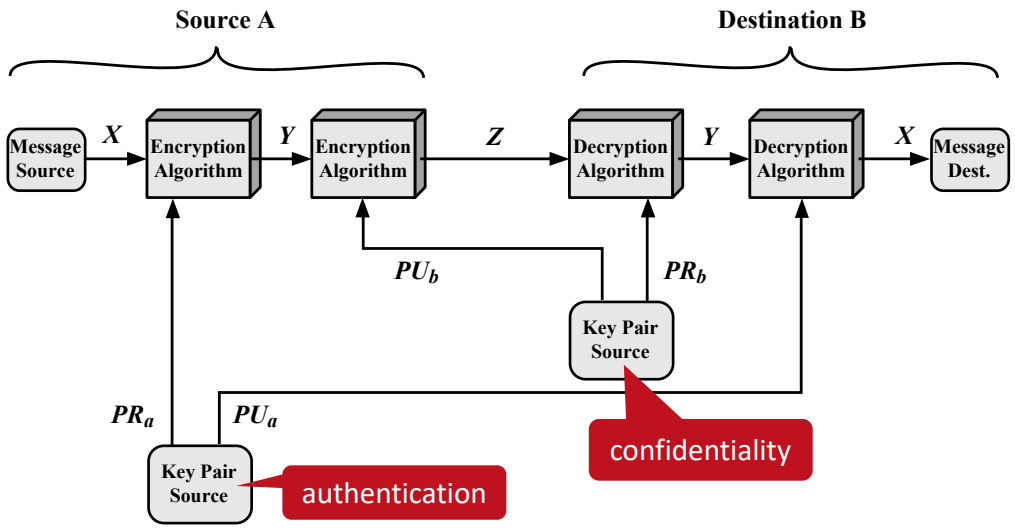
University of Antwerp
Faculty of Science

9

In this case, A prepares a message to B and encrypts it using A's private key before transmitting it. B can decrypt the message using A's public key. Because the message was encrypted using A's private key, only A could have prepared the message. Therefore, the entire encrypted message serves as a **digital signature**. In addition, it is impossible to alter the message without access to A's private key, so the message is authenticated both in terms of source and in terms of data integrity.

In this scheme, the entire message is encrypted, which, although validating both author and contents, requires a great deal of storage. Each document must be kept in plaintext to be used for practical purposes. A copy also must be stored in ciphertext so that the origin and contents can be verified in case of a dispute. A more efficient way of achieving the same results is to encrypt a small block of bits that is a function of the document. Such a block, called an authenticator, must have the property that it is infeasible to change the document without changing the authenticator. If the authenticator is encrypted with the sender's private key, it serves as a signature that verifies origin, content, and sequencing.

Combining authentication and confidentiality



It is possible to provide both the authentication function and confidentiality by a double use of the public-key scheme:

$Z = E(PU_b, E(PR_a, X))$

$X = D(PU_a, D(PR_b, Z))$

In this case, we begin as before by encrypting a message, using the sender's private key. This provides the digital signature. Next, we encrypt again, using the receiver's public key. The final ciphertext can be decrypted only by the intended receiver, who alone has the matching private key. Thus, confidentiality is provided. The disadvantage of this approach is that the public-key algorithm, which is complex, must be exercised four times rather than two in each communication.

There are 3 applications for asymmetric ciphers

	Encryption	Digital signatures	Key exchange
RSA	V	V	V
Diffie-Hellman	X	X	V
ElGamal	V	V	V
Elliptic Curves	V	V	V

University of Antwerp
Faculty of Science

In broad terms, we can classify the use of **public-key cryptosystems** into three categories

- **Encryption/decryption:** The sender encrypts a message with the recipient's public key, and the recipient decrypts the message with the recipient's private key.
- **Digital signature:** The sender "signs" a message with its private key. Signing is achieved by a cryptographic algorithm applied to the message or to a small block of data that is a function of the message.
- **Key exchange:** Two sides cooperate to exchange a session key, which is a secret key for symmetric encryption generated for use for a particular transaction (or session) and valid for a short period of time. Several different approaches are possible, involving the private key(s) of one or both parties.

Some algorithms are suitable for all three applications, whereas others can be used only for one or two of these applications.

Requirements for public-key encryption*

1. It is computationally easy to generate the key pair (PU_b, PR_b)
2. It is computationally easy, given a public key PU_b and a message M, to generate the corresponding ciphertext: $C = E(PU_b, M)$
3. It is computationally easy, given the corresponding private key PR_b and a ciphertext C, to recover the plaintext $M = D(PR_b, C) = D(PR_b, E(PU_b, M))$
4. It is computationally infeasible, knowing PU_b to generate PR_b
5. It is computationally infeasible, knowing key PU_b and ciphertext C, to recover the original message M

Optional requirement

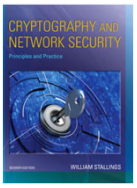
6. The two keys can be applied in either order: $M = D(PU_b, E(PR_b, M)) = D(PR_b, E(PU_b, M))$

* Similar requirements can be derived for authentication using public keys

These are formidable requirements, as evidenced by the fact that only a few algorithms (RSA, elliptic curve cryptography, Diffie–Hellman, DSS) have received widespread acceptance in the several decades since the concept of public-key cryptography was proposed.

Summary on asymmetric encryption basics

- Uses a public and a private key
- Can be used for **confidentiality** and **authentication**
- Too **computationally expensive** for encrypting large data blocks
- Three **applications**: encryption, digital signatures and key exchange
- 4 **algorithms** exist that satisfy the requirements of asymmetric encryption:
  - RSA, ElGamal, and Elliptic Curves can be used for all 3 applications
  - Diffie-Hellman can be used for key exchange



Link with the book

- Chapter 9 (Section 9.1)



Link with the videos

- Week 6 (Encryption from trapdoor permutations)

Rivest-Shamir-Adleman (RSA)

1. RSA algorithm
2. Efficient RSA operations
3. Attacks against RSA

A bit of history



Ron **R**ivest



Adi **S**hamir



Leonard **A**dleman

… jointly invented RSA in 1977 at MIT



However, **Clifford Cocks** had already invented RSA in 1973 while working for the British secret service

The information was kept classified until 1997

University of Antwerp
Faculty of Science

16

The pioneering paper by Diffie and Hellman introduced a new approach to cryptography and, in effect, challenged cryptologists to come up with a cryptographic algorithm that met the requirements for public-key systems. A number of algorithms have been proposed for public-key cryptography. Some of these, though initially promising, turned out to be breakable.

One of the first successful responses to the challenge was developed in 1977 by Ron Rivest, Adi Shamir, and Len Adleman at MIT and first published in 1978. The Rivest-Shamir-Adleman (RSA) scheme has since that time reigned supreme as the most widely accepted and implemented general-purpose approach to public-key encryption.le

RSA basics

- Each plaintext block M is represented by a number smaller than n
- The block size is therefore less than or equal to $\log_2(n) + 1$ bits
- In practice a block size of i bits is used, with $2^i \leq n \leq 2^{i+1}$

$C = M^e \bmod n$
$M = C^d \bmod n = (M^e)^d \bmod n = M^{ed} \bmod n$

The sender and receiver know n and e, and only the receiver knows d

The public key $PU = \{e, n\}$
The private key $PR = \{d, n\}$

University of Antwerp
Faculty of Science

17

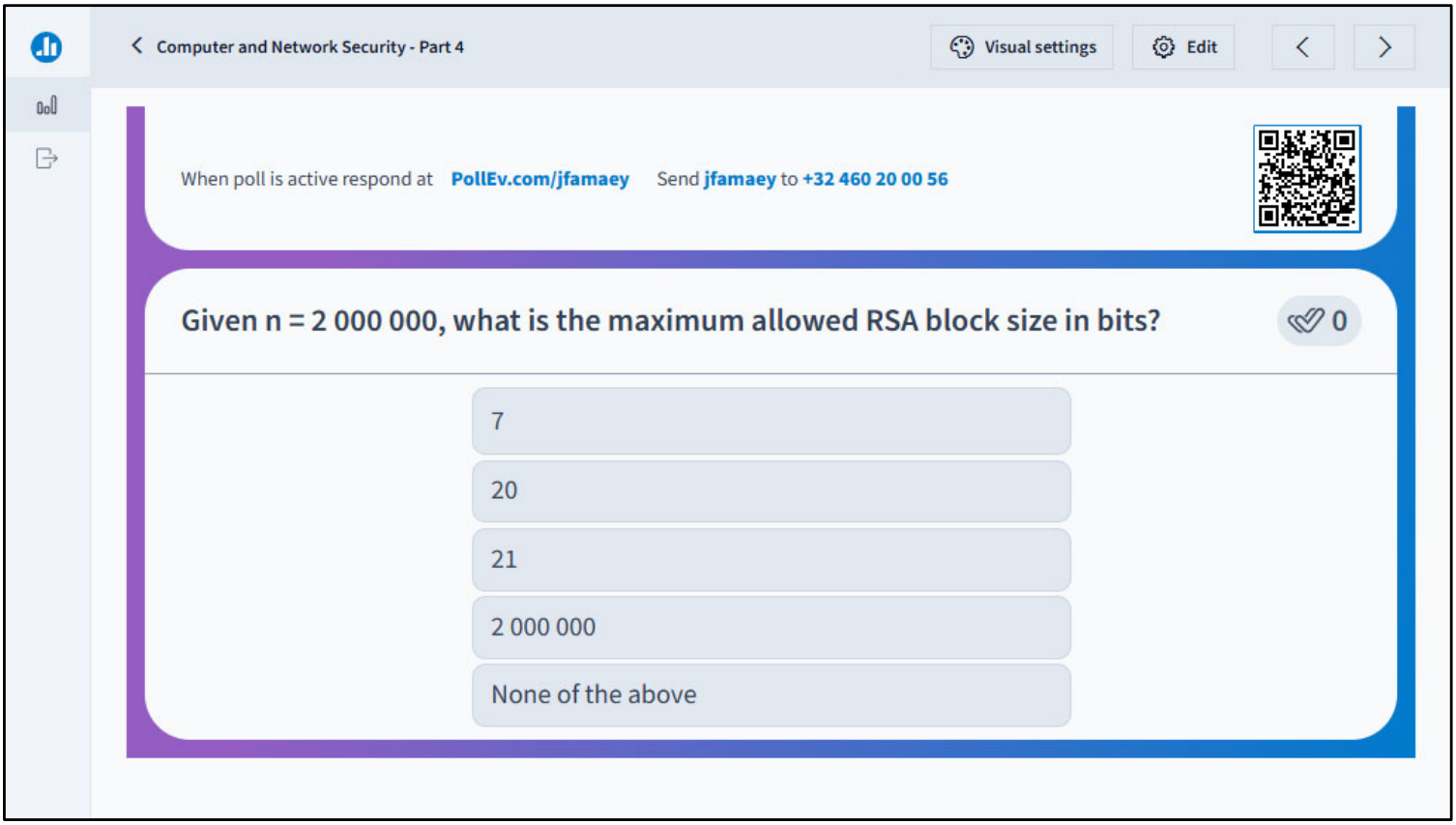
RSA makes use of an expression with exponentials. Plaintext is encrypted in blocks, with each block having a binary value less than some number *n*. That is, the block size must be less than or equal to $\log_2(n) + 1$; in practice, the block size is *i* bits, where $2^i <= n <= 2^{i+1}$. Encryption and decryption are of the following form, for some plaintext block *M* and ciphertext block *C.*

$C = M^e \bmod n$
$M = C^d \bmod n = (M^e)^d \bmod n = M^{ed} \bmod n$

Both sender and receiver must know the value of *n*. The sender knows the value of *e*, and only the receiver knows the value of *d*. Thus, this is a public-key encryption algorithm with a public key of *PU* = {*e*, *n*} and a private key of *PR* = {*d*, *n*}.

For this algorithm to be satisfactory for public-key encryption, the following requirements must be met:

1. It is possible to find values of *e*, *d*, and *n* such that $M^{ed} \bmod n = M$ for all $M < n$.
2. It is relatively easy to calculate $M^e \bmod n$ and $C^d \bmod n$ for all values of $M < n$.
3. It is infeasible to determine *d* given *e* and *n*.



Poll Title: Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity.
More info at polleverywhere.com/support

Given n = 2 000 000, what is the maximum allowed RSA block size in bits?
https://www.polleverywhere.com/multiple_choice_polls/TeQgwpkoblgTWrw?state=opened&flow=Default&onscreen=persist

Finding suitable values for e, d and n

We must find e, d and n such that: $\forall\, M < n\colon\ M^{ed} \bmod n = M$

Assuming M and n are relative prime, Euler's theorem states: $M^{\phi(n)+1} \equiv M(mod\ n)$

and given $\quad M^{ed} \bmod n = M$

$\Longrightarrow \quad ed \equiv 1(mod\ \phi(n))$

$\Longrightarrow \quad d \equiv e^{-1} \left(\bmod \phi(n)\right)$

As such, $M^{ed} \bmod n = M$ if e and d are multiplicative inverses modulo $\phi(n)$

(assuming that d and e are relative prime to $\phi(n)$, or: $\gcd(\phi(n), e) = 1$)

How can guarantee that M and n are relative prime?

- If $n = p \cdot q$, with *p* and *q* two prime numbers
- Then *M* and *n* are relative prime if M is different from *1*, *p* and *q*
- Additionally, this results in the desirable property that $\phi(n) = (p-1)(q-1)$

$\phi(n)$ = Euler totient function = Number of integers k $(1 \le k \le n)$ for which $\gcd(n, k) = 1$

University of Antwerp
Faculty of Science

20

For now, we focus on the first requirement and consider the other questions later. We need to find a relationship of the form:

$M^{ed} \bmod n = M$

The preceding relationship holds if *e* and *d* are multiplicative inverses modulo $\phi(n)$, where $\phi(n)$ is the Euler totient function (cf., Euler's theorem). It can be shown that for *p, q* prime, $\phi(pq) = (p - 1)(q - 1)$. The relationship between *e* and *d* can be expressed as:

$ed \bmod \phi(n) = 1$

This is equivalent to saying

$$ed \equiv 1 \left(\bmod \phi(n)\right)$$
$$d \equiv e^{-1} \left(\bmod \phi(n)\right)$$

That is, *e* and *d* are multiplicative inverses mod $\phi(n)$. Note that, according to the rules of modular arithmetic, this is true only if *d* (and therefore *e*) is relatively prime to $\phi(n)$. Equivalently, $\gcd(\phi(n), d) = 1$.

RSA defines e, d and n based on Euler's Theorem

The ingredients of RSA

p, q: two prime numbers	(private, chosen)
$n = pq$	(public, calculated)
e: with $\gcd(\phi(n), e) = 1; 1 < e < \phi(n)$	(public, chosen)
$d \equiv e^{-1} \pmod{\phi(n)}$	(private, calculated)

Private key $\{d, n\}$

Public key $\{e, n\}$

If n is large enough, it is infeasible to calculate p, q and $\phi(n)$

University of Antwerp
Faculty of Science

21

The private key consists of {*d*, *n*} and the public key consists of {*e*, *n*}. Suppose that user A has published its public key and that user B wishes to send the message *M* to A. Then B calculates $C = M^e \bmod n$ and transmits *C*. On receipt of this ciphertext, user A decrypts by calculating $M = C^d \bmod n$.

Exercise: Calculating RSA private key

Exercise: Given two prime numbers p = 17 and q = 11, and e = 7, what is the private {d, n} RSA key?

$n = pq$

$d \equiv e^{-1} \pmod{\phi(n)}$

r_(-1) = 160 w_(-1) = 0
r0 = 7 w0 = 1
r1 = 6 q1=22 w1 = 0-221
r2 = 1 q2 = 1

oplossing foto 28/10

Initialization

$r_{-1} = \phi(n)$ $r_0 = e$

$w_{-1} = 0$ $w_0 = 1$

Recursive algorithm

$r_i = r_{i-2} \bmod r_{i-1}$

$q_i = r_{i-2} / r_{i-1}$

$w_i = w_{i-2} - q_i w_{i-1}$

Stop condition

$if\ r_i = 1$ $then\ w_i = e^{-1}$

Exercise: Encryption with RSA

Exercise: Given p = 17, q = 11, e = 7, and d = 23, perform RSA encryption for M = 88.

$C = M^e \bmod n$
$M = C^d \bmod n = (M^e)^d \bmod n = M^{ed} \bmod n$

88^7 mod 187 = 11

Homework: Decryption with RSA



Exercise: Given p = 17, q = 11, e = 7, and d = 23, perform RSA decryption for C = 11.

$C = M^e \bmod n$
$M = C^d \bmod n = (M^e)^d \bmod n = M^{ed} \bmod n$

1. RSA algorithm
2. Efficient RSA operations
3. Attacks against RSA

Efficient exponentiation in modular arithmetic

If exponentiation is done before reduction by modulo n, the value would be gargantuan, resulting in **integer overflows**

However, the following property of modular arithmetic can be used:

$$[(a \bmod n) \times (b \bmod n)] \bmod n = (a \times b) \bmod n$$

Calculating large exponents can be made more efficient by reusing intermediary results:

$$x^{13} = (x)(x^4)(x^8)$$

We can calculate $x \bmod n$, $x^2 \bmod n$, $x^4 \bmod n$, and $x^8 \bmod n$ where each reuses the previous value

We now turn to the issue of the complexity of the computation required to use RSA. There are actually two issues to consider: encryption/decryption and key generation. Let us look first at the process of encryption and decryption and then consider key generation.

Both encryption and decryption in RSA involve raising an integer to an integer power, mod *n*. If the exponentiation is done over the integers and then reduced modulo *n*, the intermediate values would be gargantuan. Fortunately, we can make use of a property of modular arithmetic:

$[(a \bmod n) * (b \bmod n)] \bmod n = (a * b) \bmod n$

Thus, we can reduce intermediate results modulo *n*. This makes the calculation practical.

Another consideration is the efficiency of exponentiation, because with RSA, we are dealing with potentially large exponents. To see how efficiency might be increased, consider that we wish to compute x^{16}. A straightforward approach requires 15 multiplications:

$x^{16} = x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x * x$

However, we can achieve the same final result with only four multiplications if we repeatedly take the square of each partial result, successively forming (x^2, x^4, x^8, x^{16}). As another example, suppose we wish to calculate $x^{11} \bmod n$ for some integers *x* and *n*. Observe that $x^{11} = x^{1+2+8} = (x)(x^2)(x^8)$. In this case, we compute $x \bmod n$, $x^2 \bmod n$, $x^4 \bmod n$, and $x^8 \bmod n$ and then calculate $[(x \bmod n) * (x^2 \bmod n) * (x^8 \bmod n)] \bmod n$.

Algorithm for efficient modular exponentiation

Goal Find $a^b \bmod n$

b can be expressed as a binary number $b_k b_{k-1} \dots b_1 b_0$, or

$$b = \sum_{b_i \neq 0} 2^i$$

Therefore,

$$a^b = a^{\left(\sum_{b_i \neq 0} 2^i\right)} = \prod_{b_i \neq 0} a^{(2^i)}$$

$$a^b \bmod n = \left[\prod_{b_i \neq 0} a^{(2^i)}\right] \bmod n = \left[\prod_{b_i \neq 0} \left(a^{(2^i)} \bmod n\right)\right] \bmod n$$

This equation can be turned into an algorithm

$$a^b \bmod n = \left[\prod_{b_i \neq 0} a^{(2^i)}\right] \bmod n = \left[\prod_{b_i \neq 0} \left(a^{(2^i)} \bmod n\right)\right] \bmod n$$



```
f ← 1
for i ← k downto 0 do
        f ← (f · f) mod n
        if b_i = 1 then
                f ← (f · a) mod n
return f
```

$f = a^b \bmod n$

Exercise: Efficient exponentiation (1)

Exercise: Calculate $7^{21} \bmod 127$ using the efficient modular exponentiation algorithm.

```
f ← 1
for i ← k downto 0 do
        f ← (f · f) mod n
        if b_i = 1 then
                f ← (f · a) mod n
return f
```

foto 2 (kader) 28/10

Homework: Efficient exponentiation (2)

Exercise: Calculate 7^{560} mod 561 using the efficient modular exponentiation algorithm.

$f \leftarrow 1$
for $i \leftarrow k$ **downto** 0 **do**
    $f \leftarrow (f \cdot f) \bmod n$
    if $b_i = 1$ **then**
        $f \leftarrow (f \cdot a) \bmod n$
return f

Efficient operation using the public key

- A common choice for e is 65537 ($2^{16} + 1$)
- Two other simple choices are 3 and 17 (small values of e are unsafe though!)

Given the algorithm for modular exponentiation, it is **most efficient if *e* has few 1 bits**, each of these numbers only has 2 bits equal to 1

Small values for e are vulnerable to attack if no padding is used for M!

To speed up the operation of the RSA algorithm using the public key, a specific choice of e is usually made. The most common choice is 65537 ($2^{16} + 1$); two other popular choices are 3 and 17. Each of these choices has only two 1 bits, so the number of multiplications required to perform exponentiation is minimized.

However, with a very small public key, such as $e = 3$, RSA becomes vulnerable to a simple attack. Suppose we have three different RSA users who all use the value $e = 3$ but have unique values of n, namely (n_1, n_2, n_3). If user A sends the same encrypted message M to all three users, then the three ciphertexts are $C_1 = M^3 \bmod n_1$, $C_2 = M^3 \bmod n_2$, and $C_3 = M^3 \bmod n_3$. It is likely that n_1, n_2, and n_3 are pairwise relatively prime. Therefore, one can use the Chinese remainder theorem (CRT) to compute $M^3 \bmod (n_1\ n_2\ n_3)$. By the rules of the RSA algorithm, M is less than each of the n_i; therefore $M^3 < n_1 n_2 n_3$. Accordingly, the attacker need only compute the cube root of M^3. This attack can be countered by adding a unique pseudorandom bit string as padding to each instance of M to be encrypted.

The reader may have noted that the definition of the RSA algorithm requires that during key generation the user selects a value of e that is relatively prime to $\phi(n)$. Thus, if a value of e is selected first and the primes p and q are generated, it may turn out that $\gcd(\phi(n), e) \neq 1$. In that case, the user must reject the p, q values and generate a new p, q pair.

Efficient operation using the private key

A **small value of d is vulnerable** to brute-force attacks and cryptanalysis
The Chinese remainder theorem (CRT) can speed up calculations with large d!

We need to compute $M = C^d \bmod n$

We can define $V_p = C^d \bmod p; \quad V_q = C^d \bmod q$

Applying CRT gives us $X_p = q \times (q^{-1} \bmod p)$

$X_q = p \times (p^{-1} \bmod q)$

Calculates M about 4 times faster

$M = (V_p \times X_p + V_q \times X_q) \bmod n$

Using Fermat's theorem $V_p = C^{d \bmod (p-1)} \bmod p$

$V_q = C^{d \bmod (q-1)} \bmod q$

The quantities $d \bmod (p-1)$ and $d \bmod (q-1)$ can be pre-calculated

University of Antwerp | Faculty of Science

37

CRT says that given n = p x q, every M <= n can be represented as a tuple (m_1, m_2), where:

$m_1 = M \bmod p = (C^d \bmod n) \bmod p = C^d \bmod p = V_p$
$m_2 = M \bmod q = (C^d \bmod n) \bmod q = C^d \bmod q = V_q$

The definition of the bijection (CRT) states that:

$$X_p = \frac{n}{p} \times \left(\left(\frac{p}{n} \right) mod\ p \right) = \frac{p \times q}{p} \times \left(\left(\frac{p}{p \times q} \right) mod\ p \right) = q \times (q^{-1} mod\ p)$$

$$X_q = \frac{n}{q} \times \left(\left(\frac{q}{n} \right) mod\ q \right) = p \times (p^{-1} mod\ q)$$

The definition of the bijection further states that:

$M = X_p \times V_p + X_q \times V_q$

Simplifying V_p, based on Fermat's theorem:

$$V_p = C^d\ mod\ p = C^{(p-1)x + (d\ mod\ (p-1))}\ mod\ p = C^{(p-1)x} \times C^{d\ mod\ (p-1)}\ mod\ p$$

Fermat's theorem states that if C is a positive integer not divisible by p, then:

$C^{p-1} \equiv 1 (mod\ p)$ and by extension $C^{(p-1)x} \equiv 1^x \equiv 1 (mod\ p)$

Using this equality:

$$V_p = C^{(p-1)x} \times C^{d\ mod\ (p-1)} \equiv C^{d\ mod\ (p-1)}\ (mod\ p)$$

A similar proof can be devised for V_q

Key generation



University of Antwerp
Faculty of Science

38

Before the application of the public-key cryptosystem, each participant must generate a pair of keys. This involves the following tasks:

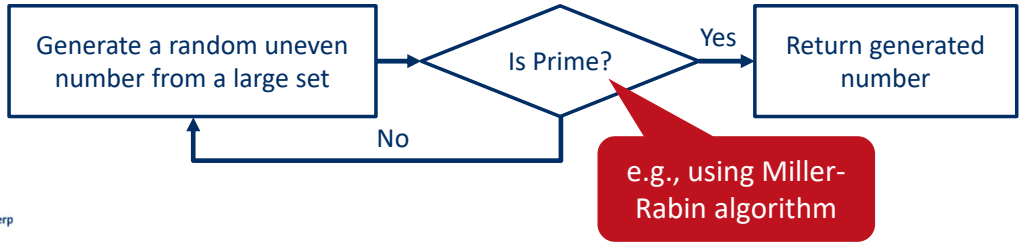
- Determining two prime numbers, *p* and *q.*
- Selecting either *e* or *d* and calculating the other.

Key generation

1 Determine two prime numbers p and q

2 Select e or d and calculate the other

- Any adversary will know $n\ (= p \times q)$
- To prevent brute force attacks, p and q should be large enough
- The method to find large p and q should be efficient



University of Antwerp | Faculty of Science

39

First, consider the selection of *p* and *q*. Because the value of *n* = *p* x *q* will be known to any potential adversary, in order to prevent the discovery of *p* and *q* by exhaustive methods, these primes must be chosen from a sufficiently large set (i.e., *p* and *q* must be large numbers). On the other hand, the method used for finding large primes must be reasonably efficient.

At present, there are no useful techniques that yield arbitrarily large primes, so some other means of tackling the problem is needed. The procedure that is generally used is to pick at random an odd number of the desired order of magnitude and test whether that number is prime. If not, pick successive random numbers until one is found that tests prime.

A variety of tests for primality have been developed. Almost invariably, the tests are probabilistic. That is, the test will merely determine that a given integer is *probably* prime. Despite this lack of certainty, these tests can be run in such a way as to make the probability as close to 1.0 as desired. As an example, one of the more efficient and popular algorithms is the Miller–Rabin algorithm. With this algorithm and most such algorithms, the procedure for testing whether a given integer *n* is prime is to perform some calculation that involves *n* and a randomly chosen integer *a*. If *n* "fails" the test, then *n* is not prime. If *n* "passes" the test, then *n* may be prime or nonprime. If *n* passes many such tests with many different randomly chosen values for *a*, then we can have high confidence that *n* is, in fact, prime.

In summary, the procedure for picking a prime number is as follows.

1. Pick an odd integer *n* at random (e.g., using a pseudorandom number generator).
2. Pick an integer *a* 6 *n* at random.
3. Perform the probabilistic primality test, such as Miller–Rabin, with *a* as a parameter. If *n* fails the test, reject the value *n* and go to step 1.
4. If *n* has passed a sufficient number of tests, accept *n*; otherwise, go to step 2.

This is a somewhat tedious procedure. However, remember that this process is performed relatively infrequently: only when a new pair (*PU*, *PR*) is needed.

It is worth noting how many numbers are likely to be rejected before a prime number is found. A result from number theory, known as the prime number theorem, states that the primes near *N* are spaced on the average one every ln(*N*) integers. Thus, on average, one would have to test on the order of ln(*N*) integers before a prime is found. Actually, because all even integers can be immediately rejected, the correct figure is ln(*N*)/2. For example, if a prime on the order of magnitude of 2^{200} were sought, then about ln(2^{200})/2 = 70 trials would be needed to find a prime.

Key generation

1 Determine two prime numbers p and q

2 Select e or d and calculate the other

Selecting e

- Select e such that $gcd(\phi(n), e) = 1$
- Calculate d such that $d \equiv e^{-1} \pmod{\phi(n)}$
  - Determining the GCD and calculating the multiplicative inverse can be done together using the **Extended Euclid's Algorithm**
  - The strategy is to select random numbers *e* (preferably with few 1-bits) until one is relative prime to $\phi(n)$

Having determined prime numbers *p* and *q*, the process of key generation is completed by selecting a value of *e* and calculating *d* or, alternatively, selecting a value of *d* and calculating *e*. Assuming the former, then we need to select an *e* such that gcd($\phi(n)$, *e*) = 1 and then calculate $d = e^{-1} \pmod{\phi(n)}$. Fortunately, there is a single algorithm that will, at the same time, calculate the greatest common divisor of two integers and, if the gcd is 1, determine the inverse of one of the integers modulo the other. The algorithm is referred to as the extended Euclid's algorithm. Thus, the procedure is to generate a series of random numbers, testing each against $\phi(n)$ until a number relatively prime to $\phi(n)$ is found. Again, we can ask the question: How many random numbers must we test to find a usable number, that is, a number relatively prime to $\phi(n)$? It can be shown easily that the probability that two random numbers are relatively prime is about 0.6; thus, very few tests would be needed to find a suitable integer.

Exercise: Selecting *e* and *d*

Exercise: Given p = 7 and q = 11, determine the highest suitable value for e and calculate d.

Relevant formulas

$n = p \times q$

$\phi(n) = (p-1)(q-1)$

$gcd(\phi(n), e) = 1$

$d \equiv e^{-1} \pmod{\phi(n)}$

Initialization

$r_{-1} = \phi(n)$ $\quad r_0 = e$

$w_{-1} = 0$ $\quad w_0 = 1$

Recursive algorithm

$r_i = r_{i-2} \bmod r_{i-1}$

$q_i = r_{i-2} / r_{i-1}$

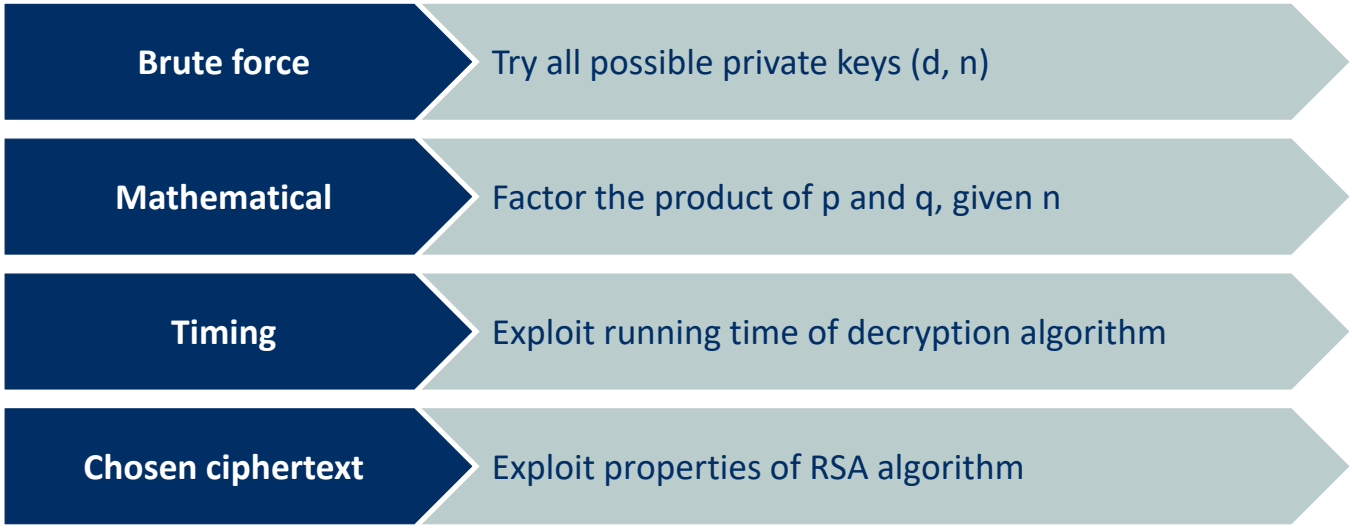
$w_i = w_{i-2} - q_i \, w_{i-1}$

Stop condition

if $r_i = 1$ $\quad$ *then* $w_i = e^{-1} = d$

1. RSA algorithm
2. Efficient RSA operations
3. Attacks against RSA

Attacks against RSA



Four possible approaches to attacking the RSA algorithm are:

- **Brute force:** This involves trying all possible private keys.
- **Mathematical attacks:** There are several approaches, all equivalent in effort to factoring the product of two primes.
- **Timing attacks:** These depend on the running time of the decryption algorithm.
- **Chosen ciphertext attacks:** This type of attack exploits properties of the RSA algorithm.

The defense against the brute-force approach is the same for RSA as for other cryptosystems, namely, to use a large key space. Thus, the larger the number of bits in *d*, the better. However, because the calculations involved, both in key generation and in encryption/decryption, are complex, the larger the size of the key, the slower the system will run.

Comparison of cipher security for brute force

Equivalence of key sizes in terms of computational effort for crypto analysis for various encryption algorithms

Symmetric Key Algorithms	Diffie-Hellman, Digital Signature Algorithm	RSA (size of *n* in bits)	ECC (modulus size in bits)
80	$L = 1024$ $N = 160$	1024	160–223
112	$L = 2048$ $N = 224$	2048	224–255
128	$L = 3072$ $N = 256$	3072	256–383
192	$L = 7680$ $N = 384$	7680	384–511
256	$L = 15{,}360$ $N = 512$	15,360	512+

Note: L = size of public key, N = size of private key

Minimum key sizes considered safe up to 2030 (November 2014 report)

Mathematical attacks

Three approaches can be identified:

1. Factor n into two primes p and q
2. Determine $\phi(n)$ directly without finding p and q
3. Determine d directly without finding $\phi(n)$

- 1 and 2 are mathematically equivalent, and easier than 3

We can identify three approaches to attacking RSA mathematically.

1. Factor *n* into its two prime factors. This enables calculation of $\phi(n)$ = (*p* - 1) * (*q* - 1), which in turn enables determination of $d = e^{-1} \pmod{\phi(n)}$.
2. Determine $\phi(n)$ directly, without first determining *p* and *q*. Again, this enables determination of $d = e^{-1} \pmod{\phi(n)}$.
3. Determine *d* directly, without first determining $\phi(n)$.

Most discussions of the cryptanalysis of RSA have focused on the task of factoring *n* into its two prime factors. Determining $\phi(n)$ given *n* is equivalent to factoring *n*. With presently known algorithms, determining *d* given *e* and *n* appears to be at least as time-consuming as the factoring problem. Hence, we can use factoring performance as a benchmark against which to evaluate the security of RSA.

RSA Laboratories key size challenges

Size of the key

Date the cipher was broken

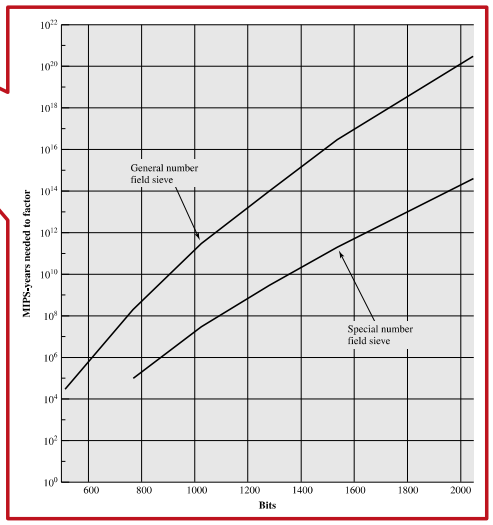
Number of Decimal Digits	Number of Bits	Date Achieved
100	332	April 1991
110	365	April 1992
120	398	June 1993
129	428	April 1994
130	431	April 1996
140	465	February 1999
155	512	August 1999
160	530	April 2003
174	576	December 2003
200	663	May 2005
193	640	November 2005
232	768	December 2009

For a large *n* with large prime factors, factoring is a hard problem, but it is not as hard as it used to be. A striking illustration of this is the following. In 1977, the three inventors of RSA dared *Scientific American* readers to decode a cipher they printed in Martin Gardner's "Mathematical Games" column. They offered a $100 reward for the return of a plaintext sentence, an event they predicted might not occur for some 40 quadrillion years. In April of 1994, a group working over the Internet claimed the prize after only eight months of work. This challenge used a public key size (length of *n*) of 129 decimal digits, or around 428 bits. In the meantime, just as they had done for DES, RSA Laboratories had issued challenges for the RSA cipher with key sizes of 100, 110, 120, and so on, digits. The latest challenge to be met is the RSA-768 challenge with a key length of 232 decimal digits, or 768 bits.

A striking fact about the progress reflected in the table concerns the method used. Until the mid-1990s, factoring attacks were made using an approach known as the quadratic sieve. The attack on RSA-130 used a newer algorithm, the generalized number field sieve (GNFS), and was able to factor a larger number than RSA-129 at only 20% of the computing effort.

Two threats to larger key sizes exist

1. Continuously improving computing power
2. Refinement of factoring algorithms



Safety precautions

- p and q should be nearly the same number of digits
- Both $(p - 1)$ and $(q - 1)$ should contain a large prime factor
- $gcd(p - 1, q - 1)$ should be small

University of Antwerp
Faculty of Science

53

The threat to larger key sizes is twofold: the continuing increase in computing power and the continuing refinement of factoring algorithms. We have seen that the move to a different algorithm resulted in a tremendous speedup. We can expect further refinements in the GNFS, and the use of an even better algorithm is also a possibility. In fact, a related algorithm, the special number field sieve (SNFS), can factor numbers with a specialized form considerably faster than the generalized number field sieve.

It is reasonable to expect a breakthrough that would enable a general factoring performance in about the same time as SNFS, or even better. Thus, we need to be careful in choosing a key size for RSA. The team that produced the 768-bit factorization observed that factoring a 1024-bit RSA modulus would be about a thousand times harder than factoring a 768-bit modulus, and a 768-bit RSA modulus is several thousands times harder to factor than a 512-bit one. Based on the amount of time between the 512-bit and 768-bit factorization successes, the team felt it to be reasonable to expect that the 1024-bit RSA moduli could be factored well within the next decade by a similar academic effort. Thus, they recommended phasing out usage of 1024-bit RSA within the next few years (from 2010).

In addition to specifying the size of n, a number of other constraints have been suggested by researchers. To avoid values of n that may be factored more easily, the algorithm's inventors suggest the following constraints on p and q:

1. p and q should differ in length by only a few digits. Thus, for a 1024-bit key (309 decimal digits), both p and q should be on the order of magnitude of 10^{75} to 10^{100}.
2. Both $(p - 1)$ and $(q - 1)$ should contain a large prime factor.

3. gcd(p - 1, q - 1) should be small.

In addition, it has been demonstrated that if $e < n$ and $d < n^{1/4}$, then d can be easily determined.

Timing attacks

- A snooper can determine the private key by keeping track of how long it takes to decipher the message
- It is applicable to all public key encryption algorithms
- It is a **ciphertext-only attack**
- For RSA it uses the timings of the modular exponentiation algorithm

Counter measures

- Constant exponentiation time (performance degradation)
- Random delay (better performance than constant time)
- Blinding (multiply ciphertext by a random number)

If one needed yet another lesson about how difficult it is to assess the security of a cryptographic algorithm, the appearance of timing attacks provides a stunning one. Paul Kocher, a cryptographic consultant, demonstrated that a snooper can determine a private key by keeping track of how long a computer takes to decipher messages. Timing attacks are applicable not just to RSA, but to other public-key cryptography systems. This attack is alarming for two reasons: It comes from a completely unexpected direction, and it is a ciphertext-only attack.

A **timing attack** is somewhat analogous to a burglar guessing the combination of a safe by observing how long it takes for someone to turn the dial from number to number. We can explain the attack using the modular exponentiation algorithm, but the attack can be adapted to work with any implementation that does not run in fixed time. In this algorithm, modular exponentiation is accomplished bit by bit, with one modular multiplication performed at each iteration and an additional modular multiplication performed for each 1 bit.

As Kocher points out in his paper, the attack is simplest to understand in an extreme case. Suppose the target system uses a modular multiplication function that is very fast in almost all cases but in a few cases takes much more time than an entire average modular exponentiation. The attack proceeds bit-by-bit starting with the leftmost bit, b_k. Suppose that the first j bits are known (to obtain the entire exponent, start with $j = 0$ and repeat the attack until the entire exponent is known). For a given ciphertext, the attacker can complete the first j iterations of the **for** loop. The operation of the subsequent step depends on the unknown exponent bit. If the bit is set, $d \leftarrow .$.

$a \mod n$ will be executed. For a few values of *a* and *d*, the modular multiplication will be extremely slow, and the attacker knows which these are. Therefore, if the observed time to execute the decryption algorithm is always slow when this particular iteration is slow with a 1 bit, then this bit is assumed to be 1. If a number of observed execution times for the entire algorithm are fast, then this bit is assumed to be 0.

In practice, modular exponentiation implementations do not have such extreme timing variations, in which the execution time of a single iteration can exceed the mean execution time of the entire algorithm. Nevertheless, there is enough variation to make this attack practical.

Although the timing attack is a serious threat, there are simple countermeasures that can be used, including the following:

- **Constant exponentiation time:** Ensure that all exponentiations take the same amount of time before returning a result. This is a simple fix but does degrade performance.
- **Random delay:** Better performance could be achieved by adding a random delay to the exponentiation algorithm to confuse the timing attack. Kocher points out that if defenders don't add enough noise, attackers could still succeed by collecting additional measurements to compensate for the random delays.
- **Blinding:** Multiply the ciphertext by a random number before performing exponentiation. This process prevents the attacker from knowing what cipher-text bits are being processed inside the computer and therefore prevents the bit-by-bit analysis essential to the timing attack.

Countering timing attacks using blinding

The operation $M = C^d \bmod n$ is replaced by:

1. Generate a random number $r \in]0, n-1]$
2. Compute $C' = C \times r^e \bmod n$, where e is the public exponent
3. Compute $M' = (C')^d \bmod n$, the ordinary RSA operation
4. Compute $M = M' \times r^{-1} \bmod n$

- Blinding results in a 2-10% performance penalty
- Given the fact that $r^{ed} \equiv r \pmod n$ it can be easily proven that this operation is indeed correct

RSA Data Security incorporates a blinding feature into some of its products. The private-key operation $M = C^d \bmod n$ is implemented as follows.

1. Generate a secret random number r between 0 and n - 1.
2. Compute $C' = C\,(r^e) \bmod n$, where e is the public exponent.
3. Compute $M' = (C')^d \bmod n$ with the ordinary RSA implementation.
4. Compute $M = M'\,r^{-1} \bmod n$. In this equation, r^{-1} is the multiplicative inverse of r mod n.

It can be demonstrated that this is the correct result by observing that $r^{ed} \bmod n = r \bmod n$. RSA Data Security reports a 2 to 10% performance penalty for blinding.

Chosen ciphertext attack (CCA)

- The attacker chooses some ciphertext and is given the corresponding plaintext in return
- The attacker chooses blocks of data that yield additional information useful for cryptanalysis

This equation no longer holds when randomly padding M before encryption

CCA example attack

Given the property $E(PU, M_1) \times E(PU, M_2) = E(PU, M_1 \times M_2)$

Given $C = M^e \bmod n$, we can derive M using CCA as follows

1. Compute $X = (C \times 2^e) \bmod n$
2. Submit X as a chosen ciphertext and receive $Y = X^d \bmod n$

$X = (C \bmod n) \times (2^e \bmod n) = (M^e \bmod n) \times (2^e \bmod n) = (2M)^e \bmod n$

Therefore, $Y = (2M) \bmod n$, from which M can be easily deduced

University of Antwerp | Faculty of Science

56

The basic RSA algorithm is vulnerable to a **chosen ciphertext attack (CCA)**. CCA is defined as an attack in which the adversary chooses a number of ciphertexts and is then given the corresponding plaintexts, decrypted with the target's private key. Thus, the adversary could select a plaintext, encrypt it with the target's public key, and then be able to get the plaintext back by having it decrypted with the private key. Clearly, this provides the adversary with no new information. Instead, the adversary exploits properties of RSA and selects blocks of data that, when processed using the target's private key, yield information needed for cryptanalysis.

A simple example of a CCA against RSA takes advantage of the following property of RSA:

$E(PU, M1) * E(PU, M2) = E(PU, [M1 * M2])$

We can decrypt $C = M^e \bmod n$ using a CCA as follows:

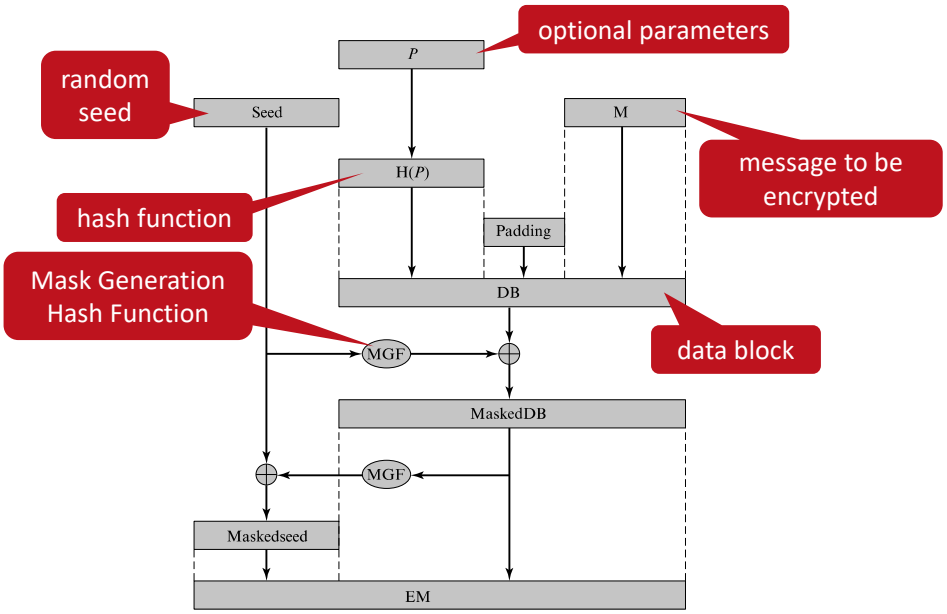
1. Compute $X = (C * 2^e) \bmod n$.
2. Submit X as a chosen ciphertext and receive back $Y = X^d \bmod n$.

But now note that:

$X = (C \bmod n) * (2^e \bmod n) = (M^e \bmod n) * (2^e \bmod n) = (2M)^e \bmod n$

Therefore, $Y = (2M) \bmod n$. From this, we can deduce M.

Counter against CCA: Optimal asymmetric encryption padding (OAEP)



University of Antwerp
Faculty of Science

57

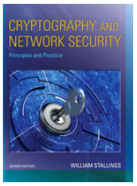
To overcome the CCA, practical RSA-based cryptosystems randomly pad the plaintext prior to encryption. This randomizes the ciphertext so that E(*PU*, M_1) * E(*PU*, M_2) = E(*PU*, [M_1 * M_2]) no longer holds. However, more sophisticated CCAs are possible, and a simple padding with a random value has been shown to be insufficient to provide the desired security. To counter such attacks, RSA Security Inc., a leading RSA vendor and former holder of the RSA patent, recommends modifying the plaintext using a procedure known as **optimal asymmetric encryption padding** (OAEP).

As a first step, the message M to be encrypted is padded. A set of optional parameters, *P*, is passed through a hash function, H. The output is then padded with zeros to get the desired length in the overall data block (DB). Next, a random seed is generated and passed through another hash function, called the mask generating function (MGF). The resulting hash value is bit-by-bit XORed with DB to produce a maskedDB. The maskedDB is in turn passed through the MGF to form a hash that is XORed with the seed to produce the maskedseed. The concatenation of the maskedseed and the maskedDB forms the encoded message EM. Note that the EM includes the padded message, masked by the seed, and the seed, masked by the maskedDB. The EM is then encrypted using RSA.

Summary on RSA

p, q:	two prime numbers; $n = p \times q$; $\phi(n) = (p-1)(q-1)$
public key $\{n, e\}$:	$\gcd(\phi(n), e) = 1$; $1 < e < \phi(n)$
private key $\{n, d\}$:	$d \equiv e^{-1} \left(\text{mod } \phi(n)\right)$
encryption:	$C = M^e \text{ mod } n$
decryption:	$M = C^d \text{ mod } n$

- **Efficient calculation** needed for modular exponentiation and key operations
- 4 main **attacks**: brute force, mathematical, timing and chosen ciphertext



Link with the book

- Chapter 9 (Section 9.2)



Link with the videos

- Week 6 (Encryption from trapdoor permutations: RSA)
- Week 6 (Encryption from trapdoor permutations: attacks)

ElGamal cryptography

ElGamal cryptography basics

- Based on discrete logarithms (like Diffie-Hellman key exchange)
- Used as a basis for the Digital Signature Algorithm (DSA) and S/MIME

Ingredients

- Global public elements
  - q prime number
  - $\alpha < q$ a primitive root of q

- Key generation
  - Generate a random integer X_A, such that $1 < X_A < q - 1$ (private key)
  - Compute $Y_A = \alpha^{X_A} \bmod q$ (public key, together with q and α)

$dlog_{\alpha,q}(Y_A) = X_A$

University of Antwerp | Faculty of Science

60

In 1984, T. Elgamal announced a public-key scheme based on discrete logarithms, closely related to the Diffie–Hellman technique. The Elgamal cryptosystem is used in some form in a number of standards including the digital signature standard (DSS) and the S/MIME email standard.

the global elements of Elgamal are a prime number q and α, which is a primitive root of q. User A generates a private/public key pair as follows:

1. Generate a random integer X_A, such that $1 < X_A < q - 1$.
2. Compute $Y_A = \alpha^{X_A} \bmod q$.
3. A's private key is X_A and A's public key is $\{q, \alpha, Y_A\}$.

The private key X_A is the discrete logarithm of Y_A base α modulo q.

ElGamal encryption and decryption

- **Encryption** (using the receiver's public key)
  - Message M is represented as an integer in the range $1 \leq M \leq q-1$
  - Choose a random integer k such that $1 \leq k \leq q-1$
  - Compute a one-time key $K = (Y_A)^k \bmod q$
  - Encrypt M as the pair of integers (C_1, C_2) with

$$C_1 = \alpha^k \bmod q, C_2 = (\mathrm{K} \times M) \bmod q$$

- **Decryption** (using the receiver's private key)
  - Recover the key by computing $K = (C_1)^{X_A} \bmod q$
  - Compute $M = (C_2 \times K^{-1}) \bmod q$

Proof that ElGamal encryption works

Step 1: Recovery of K

$K = (Y_A)^k \bmod q$ (definition of K during encryption)

$K = (\alpha^{X_A} \bmod q)^k \bmod q$ (substitution of Y_A according to definition)

$K = \alpha^{kX_A} \bmod q$ (rules of modular arithmetic)

$K = (C_1)^{X_A} \bmod q$ (substitution of C_1 according to definition)

Step 2: Recovery of the plaintext

$C_2 = (\mathrm{K} \times M) \bmod q$

$(C_2 K^{-1}) \bmod q = (\mathrm{K} \times M \times K^{-1}) \bmod q$

$= M \bmod q$

$= M$

Exercise: ElGamal encryption



Exercise: Given q = 19, and α = 10, and private key X_A = 5. What is the encrypted message (C_1, C_2) of plaintext M = 17, and k = 6?

- Key generation
  - $Y_A = \alpha^{X_A} \bmod q$
- Encryption
  - Message M is represented as an integer in the range $1 \leq M \leq q - 1$
  - Choose a random integer k such that $1 \leq k \leq q - 1$
  - Compute a one-time key $K = (Y_A)^k \bmod q$
  - Encrypt M as the pair of integers (C_1, C_2) with

$$C_1 = \alpha^k \bmod q, C_2 = (\mathrm{K} \times M) \bmod q$$

Homework: ElGamal decryption



Exercise: Given q = 19, and α = 10, public key X_A = 5, and ciphertext (11, 5), what is the corresponding plaintext M?

- Decryption
  - Recover the key by computing $K = (C_1)^{X_A} \bmod q$
  - Compute $M = (C_2 \times K^{-1}) \bmod q$

Security of ElGamal encryption: key reuse

If a message consists of multiple blocks, a **new unique k should be used** for each block, as otherwise knowledge of block M_1 can be exploited.

Given

$$C_{1,1} = \alpha^k \bmod q, C_{2,1} = (\mathrm{K} \times M_1) \bmod q$$
$$C_{1,2} = \alpha^k \bmod q, C_{2,2} = (\mathrm{K} \times M_2) \bmod q$$

Then

$$\frac{C_{2,1}}{C_{2,2}} = \frac{(\mathrm{K} \times M_1) \bmod q}{(\mathrm{K} \times M_2) \bmod q} = \frac{M_1 \bmod q}{M_2 \bmod q}$$

If M_1 is known, then M_2 can be easily calculated as follows:

$$M_2 = \left(C_{2,1}\right)^{-1} C_{2,2} M_1 \bmod q$$

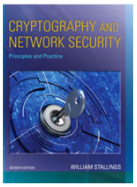
Security of ElGamal encryption: cryptoanalysis

The security of Elgamal is based on the difficulty of computing discrete logarithms:

- To recover the **private key** X_A the adversary needs to calculate
$$X_A = dlog_{\alpha,q}(Y_A)$$
- To recover the **one-time key** K, the adversary needs to calculate
$$k = dlog_{\alpha,q}(C_1)$$
- Calculating discrete logarithms is computationally infeasible if:
  - q is at least 300 decimal digits long
  - q - 1 has at least one large prime factor

Summary on ElGamal cryptography

- Private key is discrete logarithm of the public key
- Encryption and decryption use modular multiplication and exponentiation operations
- Sender calculates **unique one-time key** for each message (key-reuse is unsafe)
- Security is based on difficulty to calculate discrete logarithms



Link with the book

- [optional] Chapter 2 (Section 2.8)
- Chapter 10 (Sections 10.2)

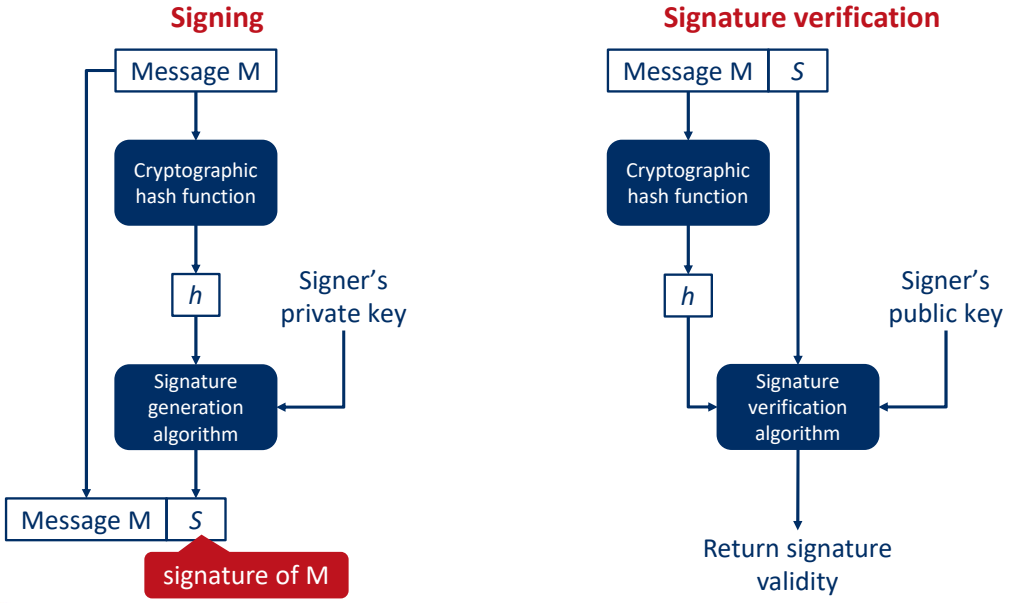


Link with the videos

- Week 6 (Public Key Encryption From Diffie-Hellman: ElGamal)

Digital Signatures

Essential elements of the digital signature process



University of Antwerp
Faculty of Science

71

The most important development from the work on public-key cryptography is the digital signature. The digital signature provides a set of security capabilities that would be difficult to implement in any other way.

Suppose that Bob wants to send a message to Alice. Although it is not important that the message be kept secret, he wants Alice to be certain that the message is indeed from him. For this purpose, Bob uses a secure hash function, such as SHA-512, to generate a hash value for the message. That hash value, together with Bob's private key serves as input to a digital signature generation algorithm, which produces a short block that functions as a **digital signature**. Bob sends the message with the signature attached. When Alice receives the message plus signature, she (1) calculates a hash value for the message; (2) provides the hash value and Bob's public key as inputs to a digital signature verification algorithm. If the algorithm returns the result that the signature is valid, Alice is assured that the message must have been signed by Bob. No one else has Bob's private key and therefore no one else could have created a signature that could be verified for this message with Bob's public key. In addition, it is impossible to alter the message without access to Bob's private key, so the message is authenticated both in terms of source and in terms of data integrity.

Required properties of a digital signature

A digital signature must

- verify the **author and time** of the signature
- **authenticate the contents** at the time of the signature
- be **verifiable by third parties**, to resolve disputes

University of Antwerp
Faculty of Science

72

Message authentication protects two parties who exchange messages from any third party. However, it does not protect the two parties against each other. Several forms of dispute between the two parties are possible.

For example, suppose that John sends an authenticated message to Mary. Consider the following disputes that could arise:

1. Mary may forge a different message and claim that it came from John. Mary would simply have to create a message and append an authentication code using the key that John and Mary share.
2. John can deny sending the message. Because it is possible for Mary to forge a message, there is no way to prove that John did in fact send the message.

Both scenarios are of legitimate concern. Here is an example of the first scenario: An electronic funds transfer takes place, and the receiver increases the amount of funds transferred and claims that the larger amount had arrived from the sender. An example of the second scenario is that an electronic mail message contains instructions to a stockbroker for a transaction that subsequently turns out badly. The sender pretends that the message was never sent.

In situations where there is not complete trust between sender and receiver, something more than authentication is needed. The most attractive solution to this problem is the digital signature. The digital signature must have the following properties:

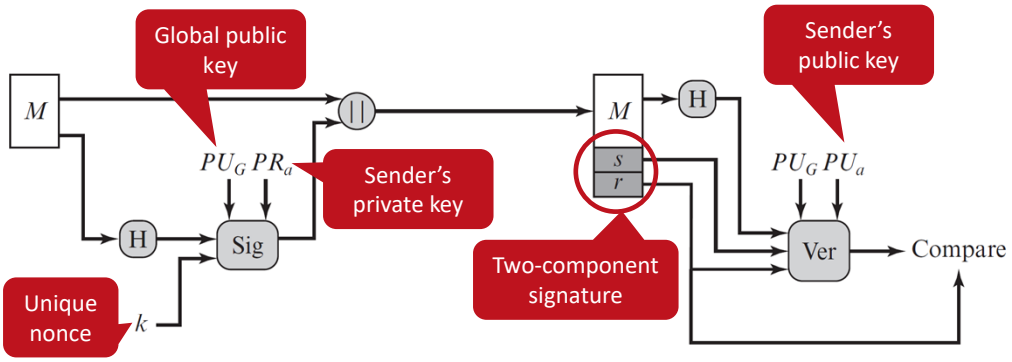
- It must verify the author and the date and time of the signature.
- It must authenticate the contents at the time of the signature.

- It must be verifiable by third parties, to resolve disputes.

Thus, the digital signature function includes the authentication function.

NIST Digital Signature Algorithm (DSA)

- DSA was first standardized in 1991 as FIPS 186
- An improved version was standardized in 1993, 1996, 2000, 2009 and 2013
  - Originally based on ElGamal/Schnorr encryption (discrete logarithms) and SHA
  - Latest version supports RSA (RSA-PSS) and elliptic curve variants



The National Institute of Standards and Technology (NIST) has published Federal Information Processing Standard FIPS 186, known as the Digital Signature Algorithm (DSA). The DSA makes use of the Secure Hash Algorithm (SHA). The DSA was originally proposed in 1991 and revised in 1993 in response to public feedback concerning the security of the scheme. There was a further minor revision in 1996. In 2000, an expanded version of the standard was issued as FIPS 186-2, subsequently updated to FIPS 186-3 in 2009, and FIPS 186-4 in 2013. This latest version also incorporates digital signature algorithms based on RSA and on elliptic curve cryptography.

The DSA uses an algorithm that is designed to provide only the digital signature function. Unlike RSA, it cannot be used for encryption or key exchange. Nevertheless, it is a public-key technique. The hash code is provided as input to a signature function along with a random number k generated for this particular signature. The signature function also depends on the sender's private key (PR_a) and a set of parameters known to a group of communicating principals. We can consider this set to constitute a global public key (PU_G). The result is a signature consisting of two components, labelled s and r.

At the receiving end, the hash code of the incoming message is generated. The hash code and the signature are inputs to a verification function. The verification function also depends on the global public key as well as the sender's public key (PU_a), which is paired with the sender's private key. The output of the verification function is a value that is equal to the signature component r if the signature is valid. The signature function is such that only the sender, with knowledge of the private key, could have produced the valid signature.

produced the valid signature.

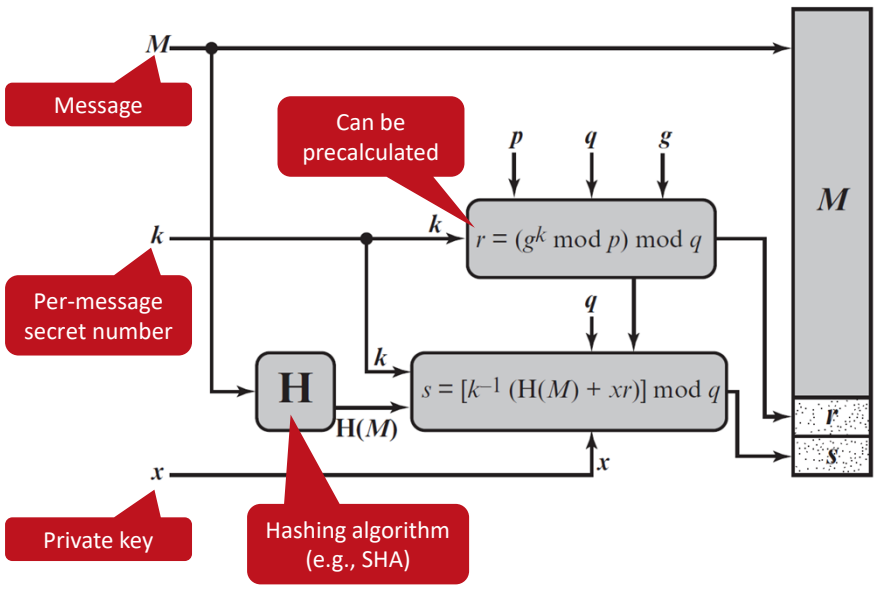
DSA key generation

- **Global public-key** components (PU_G)
  - p — prime number of length L bits (with L a multiple of 64 and at least 512)
  - q — prime divisor of $(p - 1)$ of N bits
  - h — an integer between 1 and $(p - 1)$, commonly $h = 2$ is used
  - $g = h^{(p-1)/q} \bmod p$ — with $g > 1$ (otherwise, try again with different h)
  - $PU_G = (p, q, g)$ — can be freely shared among all users of the system
- User's **private key**
  - x — (pseudo)random integer with $0 < x < q$
- User's **public key**
  - $y = g^x \bmod p$

There are three parameters that are public and can be common to a group of users. An *N*-bit prime number *q* is chosen. Next, a prime number *p* is selected such that *q* divides (*p* - 1). The original DSA constrained the length (in bits) L of p to be a multiple of 64 between 512 and 1024 inclusive. NIST 800-57 recommends lengths of 2048 (or 3072) for keys with security lifetimes extending beyond 2010 (or 2030). Finally, *g* is chosen to be of the form $h^{(p-1)/q}$ mod *p*, where *h* is an integer between 1 and (*p* - 1) with the restriction that *g* must be greater than 1. Thus, the global public-key components of DSA are the same as in the Schnorr signature scheme.

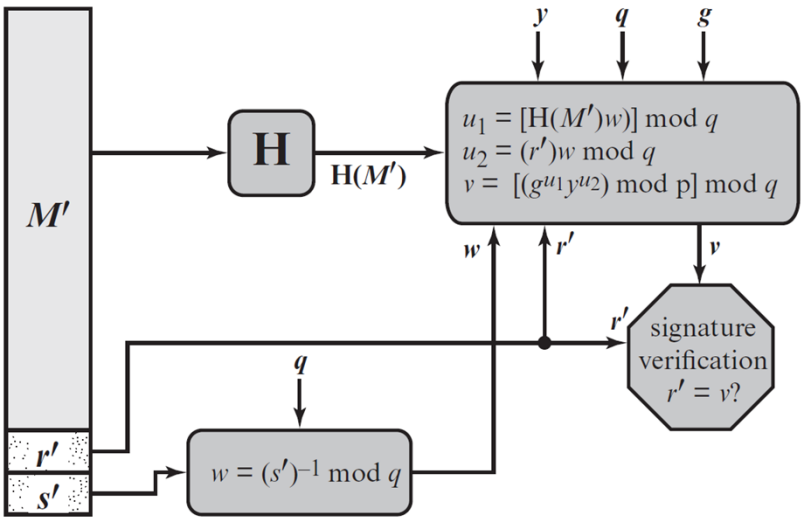
With these parameters in hand, each user selects a private key and generates a public key. The private key *x* must be a number from 1 to (*q* - 1) and should be chosen randomly or pseudorandomly. The public key is calculated from the private key as $y = g^x$ mod *p*. The calculation of *y* given *x* is relatively straightforward. However, given the public key *y*, it is believed to be computationally infeasible to determine *x*, which is the discrete logarithm of *y* to the base *g*, mod *p.*

DSA signature generation



The signature of a message *M* consists of the pair of numbers *r* and *s*, which are functions of the public key components (*p, q, g*), the user's private key (*x*), the hash code of the message H(*M*), and an additional integer *k* that should be generated randomly or pseudorandomly and be unique for each signing.

DSA signature verification



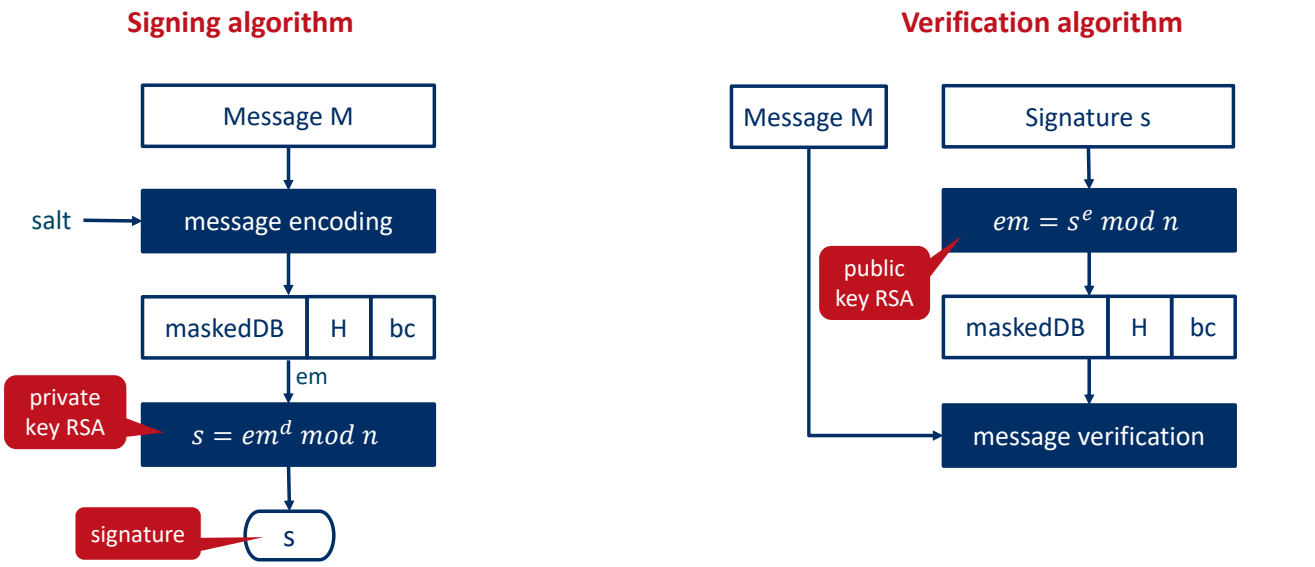
Proof that r = v is provided in Appendix K of Stallings

Let M, r', and s' be the received versions of M, r, and s, respectively. Verification is performed using the formulas shown in the figure. The receiver generates a quantity v that is a function of the public key components, the sender's public key, the hash code of the incoming message, and the received versions of r and s. If this quantity matches the r component of the signature, then the signature is validated.

Note that the test at the end is on the value r, which does not depend on the message at all. Instead, r is a function of k and the three global public-key components. The multiplicative inverse of k (mod q) is passed to a function that also has as inputs the message hash code and the user's private key. The structure of this function is such that the receiver can recover r using the incoming message and signature, the public key of the user, and the global public key.

Given the difficulty of taking discrete logarithms, it is infeasible for an opponent to recover k from r or to recover x from s. Another point worth noting is that the only computationally demanding task in signature generation is the exponential calculation $g^k \bmod p$. Because this value does not depend on the message to be signed, it can be computed ahead of time. Indeed, a user could precalculate a number of values of r to be used to sign documents as needed. The only other somewhat demanding task is the determination of a multiplicative inverse, k^{-1}. Again, a number of these values can be precalculated.

RSA-PSS: Digital signatures based on RSA



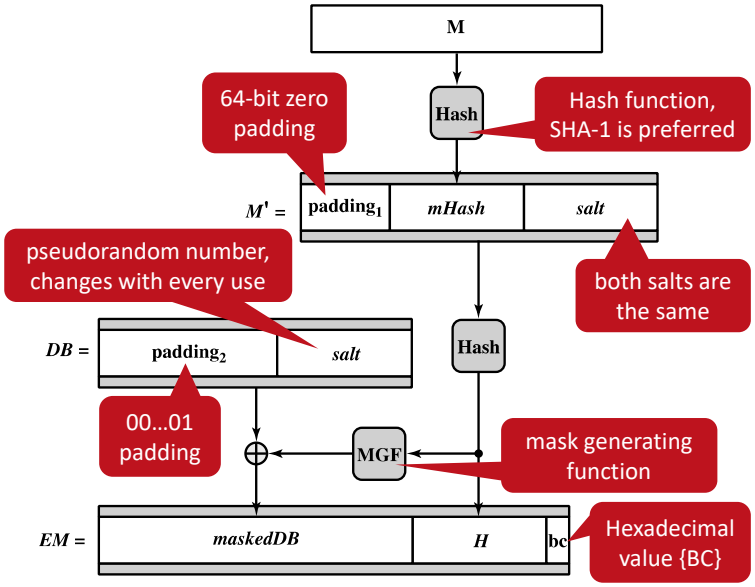
In addition to the NIST Digital Signature Algorithm and ECDSA, the 2009 version of FIPS 186 also includes several techniques based on RSA, all of which were developed by RSA Laboratories and are in wide use. We discuss the RSA Probabilistic Signature Scheme (RSA-PSS), which is the latest of the RSA schemes and the one that RSA Laboratories recommends as the most secure of the RSA schemes.

We show how the signature is formed by a signer with private key {*d*, *n*} and public key {*e*, *n*}. Treat the octet string *EM* as an unsigned, nonnegative binary integer *m*. The signature *s* is formed by encrypting *m* as follows:

$$s = m^d \bmod n$$

Let *k* be the length in octets of the RSA modulus *n*. For example, if the key size for RSA is 2048 bits, then *k* = 2048/8 = 256. Then convert the signature value *s* into the octet string *S* of length *k* octets.

RSA-PSS: Message encoding algorithm



University of Antwerp | Faculty of Science

78

The first stage in generating an RSA-PSS signature of a message *M* is to generate from *M* a fixed-length message digest, called an encoded message. We define the following parameters and functions:

- **Options**
  - **Hash:** hash function with output *hLen* octets. The current preferred alternative is SHA-1, which produces a 20-octet hash value.
  - **MGF:** mask generation function. The current specification calls for MGF1.
  - **sLen:** length in octets of the salt. Typically *sLen* = *hLen*, which for the current version is 20 octets.
- **Input**
  - **M:** message to be encoded for signing.
  - **emBits:** This value is one less than the length in bits of the RSA modulus *n*.
- **Output**
  - **EM:** encoded message. This is the message digest that will be encrypted to form the digital signature.
- **Parameters**
  - ***emLen:*** length of *EM* in octets = $\lceil emBits / 8 \rceil$.
  - **padding$_1$:** hexadecimal string 00 00 00 00 00 00 00 00; that is, a string of 64 zero bits.
  - **padding$_2$:** hexadecimal string of 00 octets with a length (*emLen* - *sLen* - *hLen* - 2) octets, followed by the hexadecimal octet with value 01.
  - **salt:** a pseudorandom number.
  - **bc:** the hexadecimal value BC.

The encoding process consists of the following steps.

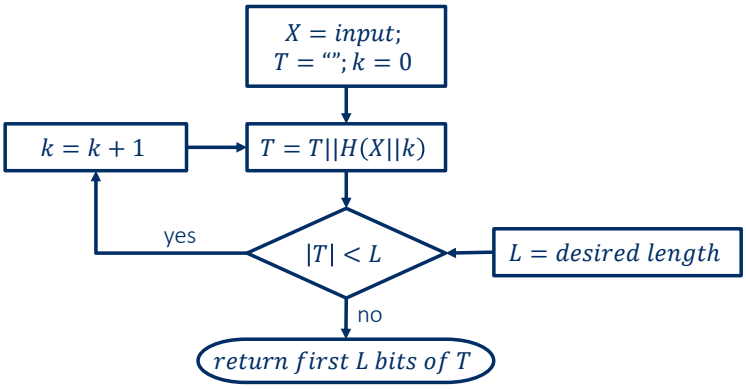
1. Generate the hash value of *M*: *mHash* = Hash(*M*)
2. Generate a pseudorandom octet string *salt* and form block *M*′ = padding_1 || *mHash* || *salt*
3. Generate the hash value of *M*′: *H* = Hash(*M*′)
4. Form data block *DB* = padding_2 || *salt*
5. Calculate the MGF value of *H*: *dbMask* = MGF(*H*, *emLen* - *hLen* - 1)
6. Calculate *maskedDB* = *DB* $\oplus$ *dbMsk*
7. Set the leftmost 8 *emLen* - *emBits* bits of the leftmost octet in *maskedDB* to 0
8. *EM* = *maskedDB* || *H* || 0xbc

We make several comments about the complex nature of this message digest algorithm. All of the RSA-based standardized digital signature schemes involve appending one or more constants (e.g., padding_1 and padding_2) in the process of forming the message digest. The objective is to make it more difficult for an adversary to find another message that maps to the same message digest as a given message or to find two messages that map to the same message digest. RSA-PSS also incorporates a pseudorandom number, namely the salt. Because the salt changes with every use, signing the same message twice using the same private key will yield two different signatures. This is an added measure of security.

RSA-PSS: Mask generating function (MGF)

Goal

Generate a cryptographically secure variable-length hash code of length L using a fixed-length output cryptographic hash function H (e.g., SHA-1)



Before explaining the RSA-PSS operation, we need to describe the mask generation function (MGF) used as a building block. MGF(*X*, *maskLen*) is a pseudorandom function that has as input parameters a bit string *X* of any length and the desired length *L* in octets of the output. MGFs are typically based on a secure cryptographic hash function such as SHA-1. An MGF based on a hash function is intended to be a cryptographically secure way of generating a message digest, or hash, of variable length based on an underlying cryptographic hash function that produces a fixed-length output.

The MGF function used in the current specification for RSA-PSS is MGF1, with the following parameters:

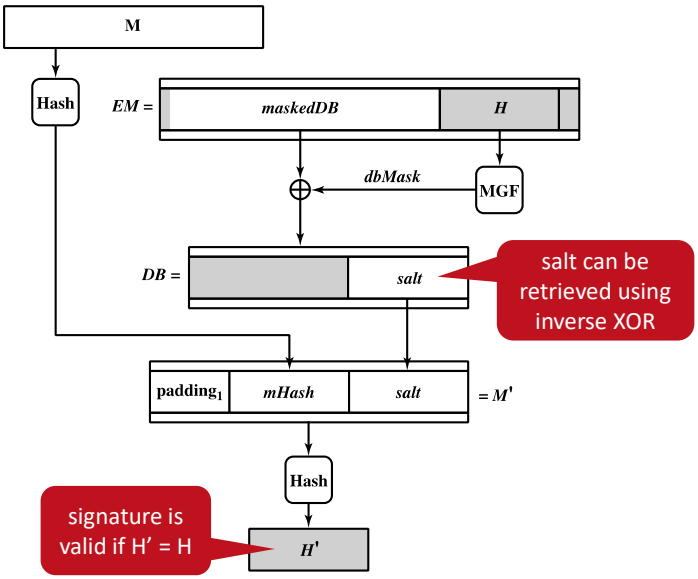
- **Options**
  - **Hash:** hash function with output hLen octets
- **Input**
  - **X:** octet string to be masked
  - **maskLen L:** length in octets of the mask
- **Output**
  - **mask:** an octet string of length maskLen

In essence, MGF1 does the following. If the length of the desired output is equal to the length of the hash value (*maskLen* = *hLen*), then the output is the hash of the input value *X* concatenated with a 32-bit counter value of 0. If *maskLen* is greater than *hLen*, the MGF1 keeps iterating by hashing *X* concatenated with the counter and appending that to the current string *T*. So that the output is

Hash (*X* || 0) || Hash (*X* || 1) || … || Hash(*X* || *k*)

This is repeated until the length of *T* is greater than or equal to *maskLen*, at which point the output is the first *maskLen* octets of *T*.

RSA-PSS: Signature verification algorithm



University of Antwerp | Faculty of Science

80

For signature verification, treat the signature *S* as an unsigned, nonnegative binary integer *s*. The message digest *m* is recovered by decrypting *s* as follows:

$m = s^e \bmod n$

Then, convert the message representative *m* to an encoded message *EM* of length $emLen = \lceil (modBits - 1)/8 \rceil$ octets, where *modBits* is the length in bits of the RSA modulus *n*.

The EM verification parameters can be described as follows:

- **Options**
  - **Hash:** hash function with output *hLen* octets.
  - **MGF:** mask generation function.
  - ***sLen:*** length in octets of the salt.
- **Input**
  - ***M:*** message to be verified.
  - ***EM:*** the octet string representing the decrypted signature, with length $\mathrm{em}Len = \lceil emBits/8 \rceil$.
  - ***emBits:*** This value is one less than the length in bits of the RSA modulus *n*.
- **Parameters**
  - **$\mathrm{padding}_1$:** hexadecimal string 00 00 00 00 00 00 00 00; that is, a string of 64 zero bits.
  - **$\mathrm{padding}_2$:** hexadecimal string of 00 octets with a length (*emLen* - *sLen* - *hLen* - 2) octets, followed by the hexadecimal octet with value 01.

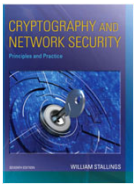
Subsequently, *EM* verification can be described as follows:

1. Generate the hash value of *M*: *mHash* = Hash(*M*)
2. If *emLen* < *hLen* + *sLen* + 2, output "inconsistent" and stop
3. If the rightmost octet of *EM* does not have hexadecimal value BC, output "inconsistent" and stop
4. Let *maskedDB* be the leftmost *emLen* - *hLen* - 1 octets of *EM*, and let *H* be the next *hLen* octets
5. If the leftmost 8 *emLen* - *emBits* bits of the leftmost octet in *maskedDB* are not all equal to zero, output "inconsistent" and stop
6. Calculate *dbMask* = MGF (*H*, *emLen* - *hLen* - 1)
7. Calculate *DB* = *maskedDB* $\oplus$ *dbMsk*
8. Set the leftmost 8 *emLen* - *emBits* bits of the leftmost octet in *DB* to zero
9. If the leftmost (*emLen* - *hLen* - *sLen* - 1) octets of *DB* are not equal to padding_2, output "inconsistent" and stop
10. Let *salt* be the last *sLen* octets of *DB*
11. Form block *M*' = padding1 || *mHash* || *salt*
12. Generate the hash value of *M*': H' = Hash(*M*')
13. If *H* = *H*', output "consistent." Otherwise, output "inconsistent"

The shaded boxes labeled *H* and *H*' correspond, respectively, to the value contained in the decrypted signature and the value generated from the message *M* associated with the signature. The remaining three shaded areas contain values generated from the decrypted signature and compared to known constants. We can now see more clearly the different roles played by the constants and the pseudorandom value *salt*, all of which are embedded in the *EM* generated by the signer. The constants are known to the verifier, so that the computed constants can be compared to the known constants as an additional check that the signature is valid (in addition to comparing *H* and *H*'). The salt results in a different signature every time a given message is signed with the same private key. The verifier does not know the value of the salt and does not attempt a comparison.

Summary on digital signatures

- Protect two communicating parties from opponents as well as each other
  - Message content authentication
  - Non-repudiation of the source
- Based on public key encryption
  - NIST Digital Signature Algorithm (DSA): Based on ElGamal and SHA
  - RSA-PSS: Based on SHA and RSA



Link with the book

- Chapter 13 (Sections 13.1, 13.4, 13.6)

End of Part 4

University of Antwerp
Faculty of Science